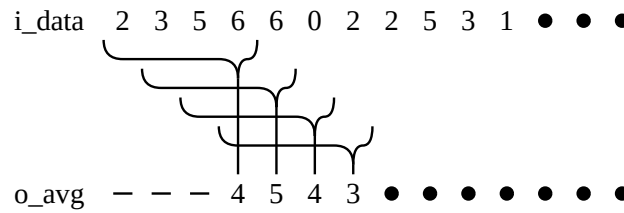


2.11 Example: Moving Average

In this section we will design a circuit that performs a moving average as it receives a stream of data. When each new data item is received, the output is the average of the four most recently received data.



2.11.1 Requirements and Environmental Assumptions

1. Input data is sent sporadically, with at least 2 clock cycles of bubbles (invalid data) between valid data.
2. When the input data is valid, the signal `i_valid` is asserted for exactly one clock cycle.
3. Input data will be 8-bit signed numbers.
4. When output data is ready, `o_valid` shall be asserted.
5. The output data (`o_avg`) shall be the average of the four most recently received input data. Output numbers shall be truncated to integer values.

2.11.2 Algorithm

We begin by exploring the system mathematically. If we had an input stream of data (x) indexed by time (e.g. x_t is the value of x at time t) and we received a new piece of data in each clock cycle, the equation for the output would be:

$$avg_t = (x_{t-3} + x_{t-2} + x_{t-1} + x_t) / 4$$

If we look at the equation of avg over time, we see that part of the calculations that are done for time t are the same as those for $t + 1$:

$$\begin{aligned} avg_5 &= (x_2 + x_3 + x_4 + x_5) / 4 \\ avg_6 &= (x_3 + x_4 + x_5 + x_6) / 4 \\ avg_7 &= (x_4 + x_5 + x_6 + x_7) / 4 \end{aligned}$$

We can reduce the number of computations by keeping a running sum of the four most recent data. At each clock cycle, we subtract the oldest value from the sum and add the new value:

$$\begin{aligned}
sum_5 &= sum_4 - x_1 + x_5 \\
sum_6 &= sum_4 - x_2 + x_6 \\
sum_7 &= sum_4 - x_3 + x_7
\end{aligned}$$

The average is then computed simply dividing the sum by the number of entries in the sliding window (4 in our case).

$$avg = sum/4$$

Generalizing the above to time t , we have:

$$\begin{aligned}
sum_{t+1} &= sum_t - x_{t-4} + x_t \\
avg_t &= sum_t/4
\end{aligned}$$

Notice that we write the equation for the sum with $t + 1$ on the left-hand-side, rather than t . This indicates that sum will be stored in a register, which is needed because sum_{t+1} is defined in terms of sum_t .

From the equation for sum , we see that we need to store four values of the input data x .

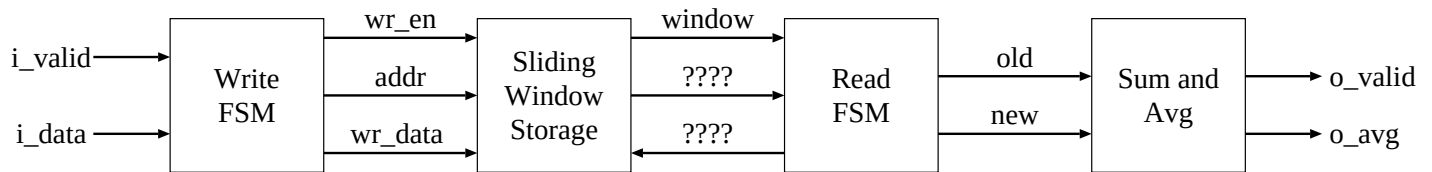
2.11.3 Top-Level Block Diagram

Summary of system behaviour deduced from exploring requirements and algorithm:

1. Read in `i_data` each time that `i_valid` is '1'.
2. Store sliding window of four most recent values of `i_data`.
3. Read the oldest data value (*old*) from the sliding window.
4. Update sum with $sum_t - old + new$ where *new* is the new data to be written to the window.

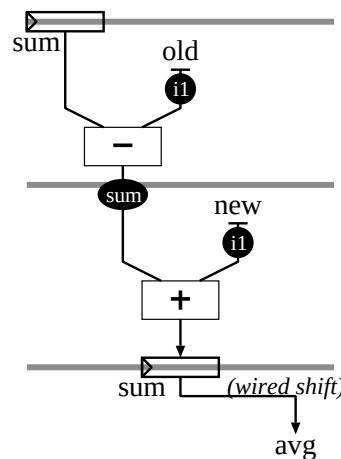
The system-level block diagram represents the top-level structure. To create a system-level block diagram, we need to analyze the different tasks to see whether each task should be done by a separate module, or whether multiple tasks should be performed by the same module.

The different tasks in the system-level behaviour communicate with each other, but don't require the same hardware resources for either computation or storage. We choose to allocate each task to a separate module. Because the system is so simple, we can already identify most of the major signals for communicating between the different modules.



We now proceed to design each of the different modules. Often, each module can be designed in parallel. In this case though, we don't yet know how we will implement the storage or the sum and average calculation. The behaviour of the Read FSM and Write FSM modules will be dictated by the storage and the sum and average computation. We choose to start with the sum and average computation, because dataflow diagrams give us a structured approach for doing the design.

2.11.4 Dataflow Diagram



We chose to put the subtraction and addition operations in separate clock cycles because we hypothesize that a single adder/subtractor will require less area than one adder and one subtractor. Before proceeding further with this design, we should synthesize these components to test our hypothesis. It would be quite disappointing to “optimize” our entire design based on a hypothesis that turns out to be false!

In our system-level block diagram, the signals from the Read-FSM to the Sum-and-Avg were: `old` and `new`. Now that we have designed dataflow diagram for the Sum-and-Avg, we see that we need only one signal, because we will send the old data in one clock cycle and the new data in the next clock cycle. We will also need a control signal to tell the statemachine for the Sum-and-Avg when to start the computation. In section 2.11.9, we will redraw the system-level block diagram and update it with changes that we made during the design process.

2.11.5 Storage

We have only four items to store, so we will use four registers, rather than a memory array. For less than sixteen items, registers are generally cheaper. For sixteen items, the choice between

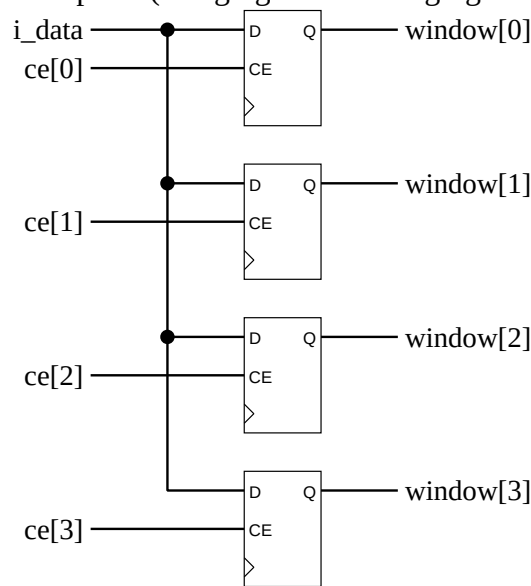
registers and a memory array is highly dependent on the design goals (e.g. speed vs area) and implementation technology.

Two ways to implement a sliding window:

shift-register Each time new data is loaded $R[0] = new$ and $R[i] = R[i - 1]$.

circular buffer A state machine keeps track of the position of the oldest piece of data. Each time new data is loaded, the new data overwrites the oldest piece of data and all of the other registers do not change. The state machine increments to point to the next register, which now olds the oldest piece of data.

Despite the apparent simplicity of the shift-register approach, the circular buffer design usually requires less area. Although we will not cover power consumption until chapter??, a huge advantage of the circular buffer approach is that fewer registers change value in each clock cycle, which greatly reduces the power consumption (charging and discharging of capacitors).



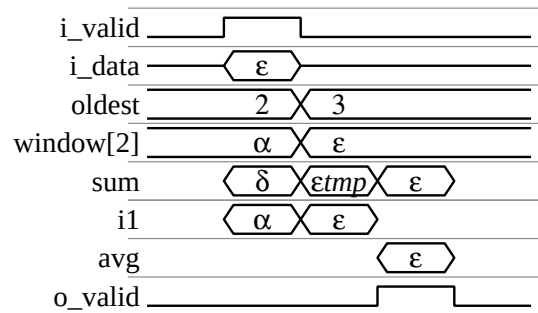
The Write-FSM tells the Sliding Window Storage when to write data, the address to write to, and the data to write. In our system-level block diagram, the signals between these modules were: `wr_en`), `addr` and `wr_data`. Now that we have designed the storage module, we see that rather than a write-enable and address signal, the actual signals we need are four chip-enable signals. We will update the system-level block diagram in section 2.11.9.

2.11.6 Timing Diagram

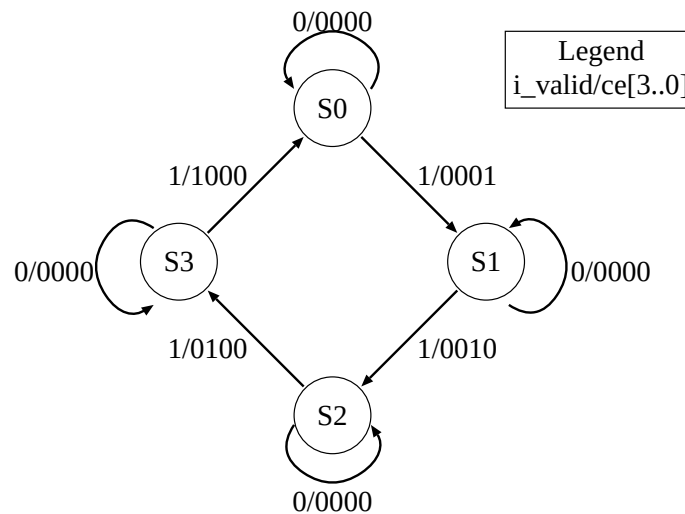
To design the state machines, we need to understand the clock-cycle by clock-cycle behaviour of the system.

The signal `oldest` holds the index of the register with the oldest piece of data.

sequence of input data: α β γ δ ϵ



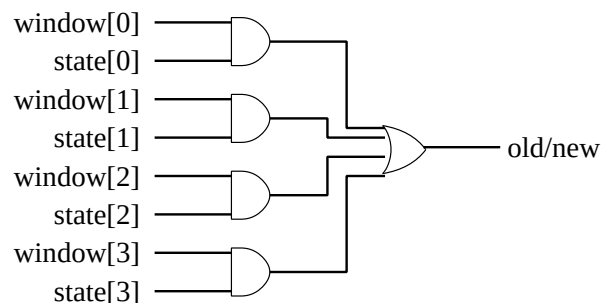
2.11.7 Write State Machine



Use a one hot encoding for the states. The chip-enable signals are then just the conjunction of the state bits with **i_valid**.

2.11.8 Read State Machine

For the read state machine, reuse the Write-FSM to drive mux-selectors. Because we have a one-hot encoding, don't use normal multiplexors, which take a full binary encoded select signal. Instead, use a 4:1 decoded mux, which is just four AND gates followed by a 4-input OR gate.

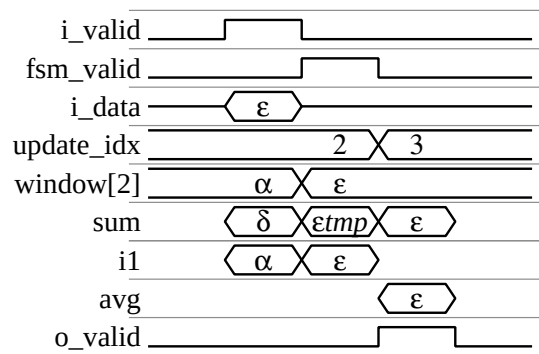


Based on the behaviour of the sliding-window storage in the timing diagram, we see that in the clock cycle when `i_valid` is asserted, `window[2]` has the oldest data (α) and in the next clock cycle, this same register has the new data. However, in the timing diagram, the signal `oldest` is changes in the same clock cycle as the new data appears in the window.

For area efficiency, we would like to use the same circuitry to read the old and new data from the window in back-to-back clock cycles. We need to delay the update of `oldest` by one clock cycle, which we can do by inserting a register between the input `i_valid` and the `valid` signal that drives the state machine.

The updated timing diagram is shown below, where we renamed `oldest` to `update_idx` to represent the new purpose of the signal.

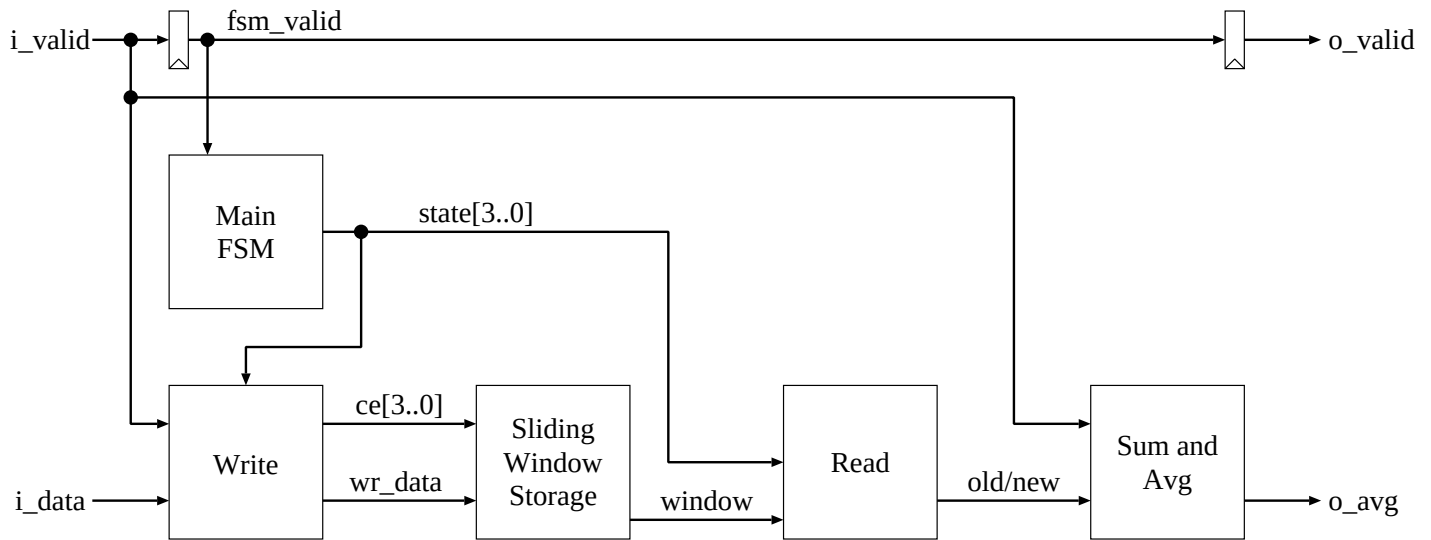
sequence of input data: α β γ δ ϵ



Adjusting in which clock cycle a signal is used is common in hardware design. There is often a choice as to which signal to change, where each choice has tradeoffs between area, timing, and simplicity. For this system, two alternative choices are to use an additional register to store the new data or use an additional register to store a delayed version of `oldest`. A register for the new data would be eight bits wide and a register for `oldest` would be four bits wide. Each of these options is more expensive than the one-big flip-flop that is needed for the delayed version of `i_valid`.

Because we are reusing the Write-Fsm, we separate the Write-FSM from the circuitry to generate the chip-enable signals and create a Main-FSM that drives the chip-enable generation (Write module) and the Read module.

2.11.9 Putting It All Together



From the timing diagram, we see that `i_valid` initiates the datapath computation. The signal `o_valid` is just `i_valid` delayed by two clock cycles. The state machine to control the datapath will just be a chain of flops that connect `i_valid` to `o_valid`. We can reuse `fsm_valid` for both the Main-FSM and the second flop in the chain from `i_valid` to `o_valid`.